Simulation-Based Estimation of General Structural Network Models

Cameron Fen*

November 14, 2022

Structural estimation to fit an economic model on a single graph is an open problem in economics and adjacent fields. While there are techniques to estimate structural models on a collection of iid graphs, because nodes in a network are not iid, there is little literature on a general purpose algorithm to fit a structural model on a single network–at least work that the author is aware of. This paper provides an algorithm borrowed from the deep learning literature and adopted to networks, that theoretically allows for Bayesian and likelihood estimation of arbitrary structural models using graphs as long as graph data is in the support of the networks simulated by the structural model. Tests on simulated data as well as an empirical application demonstrate the validity and the accuracy of this approach.
JEL Codes: C11, C68, C63, C45
Keywords: Neural Networks, Bayesian Inference, Network Estimation, Structural Models, Simulation-Based Estimators

*Cameron Fen is PhD student at the University of Michigan, Ann Arbor, MI, 48104 (E-mail: camfen@umich.edu. Website: cameronfen.github.io.). The author thanks Florian Gunsilius for helpful feedback. All errors are my own.

I. Introduction

Unlike structural estimation of dynamic models, structural estimation of network models is a less mature field. The presence of a graph that has predetermined connections leads to difficulty. Conventional approaches to structural estimation on networks work only in specialized settings. Often, one either must build a structural network model that generates good statistics that can be matched with data or abandon the structural paradigm and use a reduced form model like an Exponential Random Graph Model (ERGM). There is a need for a general purpose structural network estimation procedure in much the same way that Method of Simulated Moments (MSM) (McFadden, 1989) opened the doors to the construction of arbitrary structural models in stuctural macroeconomics among other fields. It is possible to use method of simulated moments in structural estimation on networks (Banerjee et al., 2013), however this requires data on many different networks with each network assumed to be generated iid from the same model. However due to greater applicability, network econometricians are more interested in estimation of structural models on a single network, but because nodes and edges are not iid, one cannot aggregate moments over nodes or edges, to perform a method of moments like approach. Thus there really aren't good methods that allow structural estimation on data consisting of a single network.

How can one estimate arbitrarily defined structural network models on a single network? Certainly in particular cases it is possible to arbitrarily tailor the model to produce statistics that can be matched with statistics in the data, but, to the best of my knowledge, a general case algorithm has not been proposed. In theory this should be possible in a Bayesian setting as each node provides additional information but the whole network is not made up of iid nodes. This bears similarity to the macroeconomic framework where time series constructed from a filtering model are not iid but rather can be viewed as a single data pointed generated by a model that has latent connections across times steps. Another difficulty includes how to define a likelihood function generated by a structural model over all generated graphs and an appropriate perturbative shock on a graph which allows a likelihood function to rank how close a specific model parameterization is to any graph found in the wild. The analogue to this would be a normal innovation, which has support, with

decreasing mass, to any numerical value from an expected value of the model. Whereas every value is in the support of a normal innovation, it is not trivial to design a shock process for graphs that have this property. Since the true data must be in the support of the model, in order to perform a likelihood based approach with graphs one has to have a structural model that assigns positive support to the data. This in practice generally implies that the model assign positive support to all graphs with the same number of nodes.

There are many papers that have implemented structural models, but due to lack of estimation tools, do not perform full estimation on real data (Calvo-Armengol and Jackson, 2004), (Carvalho and Voigtländer, 2014), (Calvó-Armengol and Jackson, 2007). Because conventional estimation techniques are generally ad-hoc or only work for data with multiple networks, many papers choose not to estimate their models on data and only report theoretical properties of their models (Jackson and Wolinsky, 1996), (Gilles, Johnson et al., 2000), (Furusawa and Konishi, 2007).

This paper introduces an algorithm that can MLE or Bayesian estimate, from simulation, almost arbitrary structural models that generate networks as long as the model can generate the true data. This assumption is the direct consequence of likelihood-based estimation. I also demonstrate empirically the model recovering the data generating parameters in a variety of cases. Additionally I apply my approach to estimate Bramoullé et al. (2012) model, which the authors were not able to estimate. This paper attempted to model the process of academic citing. When examining the maximum likelihood solution, my results mirror the Bramoullé et al. (2012) result. When using the mean of the posterior, my results indicate there is a lot of hidden homophily, which isn't just captured by counting the number of citations that learned about the work cited by an intermediary paper.

Unlike other general purpose network estimation approaches, it is efficient and not dependant on multiple networks for identification. The approach works in practice for data that contains many networks. The only practical requirement is that, like other simulation-based approaches, one can apply my approach if there is a 1-1 correspondence between model simulated network data and real network data, in addition to the widely used assumption that the embedding model, which converts the graph to a set of numbers that can be used to form a conditional density, can differentiate

between any graph simulated by the sturctural model. The approach doesn't require a likelihood and is much faster than brute force grid search, particularly when the number of parameters is large. Additionally the approach, like MSM, works on simulated data itself, estimating a likelihood without the need to construct a loss function.

II. Methods

This section will first discuss normalizing flows, then it will discuss the general approach for simulation-based estimation, the Sequential Neural Posterior Estimation (SNPE) algorithm. Finally, it will discuss embedding networks, which can be one of a graph convolutional neural network which converts graphs into sufficient statistics for the normalizing flow to train on, and FEATHER (Rozemberczki and Sarkar, 2020), an algorithm that uses the characteristic function of a random walk to create sufficient statistics. For more information on flows and SNPE algorithm see the section "Background on Simulation Neural Posterior Estimation and Metropolis Hastings" in Fen (2022)

II.A. Background on Simulation Neural Posterior Estimation (SNPE)

A discussion of the SNPE algorithm can be found in Fen (2022). As such, I will refrain from more than a brief discussion of the method. Likewise the density estimator used in SNPE is also discussed in that paper and I will mainly focus on the graph neural network and other embedding methods that convert graph data into sufficient statistics. For more information on the normalizing flow density estimator see the above paper.

The SNPE algorithm performs simulation based Bayesian inference. As such it will recover the posterior distribution. It shares similarities with the Kristensen and Shin (2012) approach to simulation-based inference. In the SNPE algorithm described in Fen (2022), one estimates a posterior by sampling from the parameters space $\theta \sim P(\theta)$, sampling from the model, $x \sim P(x|\theta)$, and fitting a density estimator of the posterior, $P(\theta|x)$, on the joint samples (x, θ). This gives a posterior estimate when $P(\theta|x = X)$ is conditioned on the real data X. Here x and X are model

simulated network data and true data respectively. Since in this case x/X is a graph, SNPE uses a graph convolutional network or a graph embedding algorithm that converts the network data into a set of sufficient statistics to be used in estimating the density $P(\theta|x = X)$.

Algorithm 1: SNPE Algorithm

Input: Simulator $p(x|\theta)$, prior $p(\theta)$, data x_0, flow $f_\phi(x|\theta)$, Rounds R, Samples S;

Initialize: Posterior $p^{(0)} = p(\theta)$, data set D = {};

for $i \leftarrow 1$ **to** R **do**

  Sample $\theta^{(n)} \sim p^{(i-1)}$ for $n = 1...S$ with Monte Carlo;

  Simulate $x^{(n)} \sim P(x|\theta^{(n)})$ for $n = 1...S$;

  Concatenate data $D = D \cup \{x^{(n)}, \theta^{(n)}\}_{n=1}^{S}$;

  while $d_\phi(x, \theta)$ *not converged* **do**

    Sample $\{x^{(i)}, \theta^{(i)}\}_i^B \sim D$ from D;

    Train $f_\phi(\theta|x)\dfrac{p^{(i-1)}}{p(\theta)}$ on $\{x^{(i)}, \theta^{(i)}\}_i^B$;

  end

  Update posterior $p^{(i)} \propto f_\phi(\theta|x)$;

end

Compared to Kristensen and Shin (2012), the normalizing flow improves upon the kernel density estimator (KDE) with the ability to estimate conditional densities, high dimensional densities, and the ability to sample and estimate without resort to rejection sampling.

Next I will discuss a normalizing flow. Briefly, a neural autoregressive normalizing flow layer is parameterized in this way:

$$y_\iota^j = \sigma(\beta_\iota^0(y_{a:\iota-1}^i) + \beta_\iota^1(y_{a:\iota-1}^i) * y_\iota^i)$$

The output of a given layer, y^j is a linear function of the β^0, β^1 and the inputs y^i, composed with a non-linearity (ie a logit transformation for example). Since y^j and y^i are assumed to be multi-dimensional, the ιs capture the indices of y^j and y^i. What makes this model flexible is that β^0 and β^1 are both the outputs of a non-parameteric model typically a neural network. If you know

all the y^is, like in the usual case where you know the inputs of a function, you can straightforwardly get all the y^js. One of the main innovations of this normalizing flow, is the ability to maintain non-parametric flexibility, while also allowing evaluations in the inverse direction. If you know all the y^js, you can get all the y^is.

I will now demonstrate how to evaluate the normalizing flow in the inverse direction. The $\beta's$ for the first element $\iota = 0$ in y^j are not conditioned on anything. However, with $\iota > 0$, the beta's are neural networks that are conditioned on $y^i_{0:\iota-1}$, iotas with an index lower than the current iota. In this way if one knows y^j one can obtain y^i through an inductive inversion process where first one solves for y^i_0 through the affine constants $\beta^{0,1}_{\iota=0}$ and then assuming the induction hypothesis that one knows all y^is up to $\iota - 1$, one knows the inputs into each of the $\beta^{0,1}_\iota$. Since now the βs can be evaluated for equation ι, one can calculate invert the now affine equation to get y^i_ι. If one wants to condition on external variables one can just append these variables as inputs to each $\beta^{0,1}$ neural network in addition to the $y^i_{0:\iota-1}$ inputs to each beta. In order to move from one layer to a normalizing flow, one then stacks the output of one layer into the input for the next layer, building a stack of neuaral autoregressive bijector layers.

This flow then learns a mapping from a base distribution that is chosen for convenience, like a normal distribution with a known mean and variance, to the target distribution. The target distribution is unknown and the density to be estimated. The mapping from the base to the target is trained in the estimation procedure. This induces a change of variables transformation in much the same way a quadratic relationship transforms a normal to a chi-squared distribution–except in this case the change of variables contains tunable parameters. The main benefit of the invertability of the flow is that one can sample from the flow, by sampling from the base distribution and feeding the sample through the flow. Likewise one can sample (or use data) from the target distribution and feed the data back through the inverse flow and evaluate the likelihood of the point being seen under the flow. This is discussed next.

Discussing how to perform maximum likelihood density estimation. Given samples from a base distribution, $p(y^a)$ with random vector realizations y^a; a normalizing flow is a function, $f(y^a)$ mapping y^a to a target random variable y^z, which has density, $q(y^z)$ such that the probability $q(y^z)$

is related to the probability $p(y^a)$ via:

$$q(y^z) = p(y^a)|det\frac{df^{-1}}{dy^a})| = p(y^z)|det\frac{df(y^a)}{dy^a}|^{-1} \tag{1}$$

This is the standard formula to calculated a change in variable given the probability of $q(y^z)$ knowing the transformed probability distribution $p(y^a)$ given the change in variable $f : y^z \to y^a$. For example when changing the variable of a normal distribution to a chi-squared distribution, one not only evaluates the probability of the chi-suqared sample by taking the square root and evaluating that under a normal distribution, one also has to apply the jacobian term to the change in measure. This is what the second term, $|det\frac{df(y^a)}{dy^a}|^{-1}$, is. The appendix of (Fen, 2022) shows in more detail how to construct a flow so that it is both invertable and can approximate any continuous distribution. Assuming both these things, one can fit this density estimators on samples by using the change of variable formula to calculate the likelihood of any point, $q(y^z)$, take derivatives with respect to the parameters of the flow (the neural networks that generate the pseudo-parameters), and perform maximum likelihood.

II.B. Graph Convolution Neural Networks

The last thing to do is to convert the graphs into numerical statistics so the normalizing flow density estimator can be conditioned on them. This section will discuss the basics of feed forward neural networks and then graph convolutional neural networks.

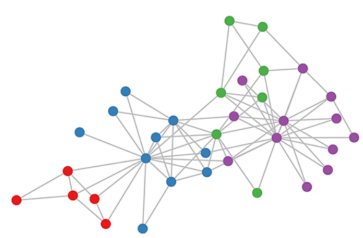
A layer in a feed forward neural network is defined as:

$$x^j = \sigma(A_i x^i + B_i) \tag{2}$$

Here σ is a nonlinearity like a logistic link function, but it could be something as simple of a rectified linear unit (ReLU) (Agarap, 2018). Here A is a matrix that maps $x^i \in R^n$ to $x^j \in R^m$. If the link function was a logistic, a single layer would be a logistic regression with multiple outputs. What gives a feed forward neural network power is the ability to compose one layer with another.

The output of a lower layer becomes the input to the higher layer. Hornik, Stinchcombe and White (1989) showed that a feed forward neural network with a single hidden (intermediate) layer is enough to approximate any continuous distribution in the infinite width limit of the hidden layer.

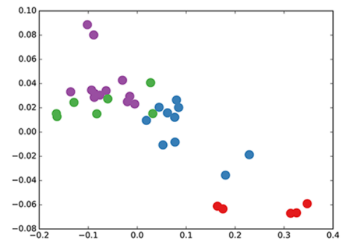
Next I will discuss graph convolutional neural networks (GCNN) (Kipf and Welling, 2016). In particular, define an adjacency matrix (or adjacency-like matrix like the Laplacian) as A. Typically normalizing the matrix by dividing by the degree matrix is helpful. Call the normalized matrix A^*. Define x^0 as the matrix of node characteristics. Then $X^j = \sigma(W_i x^i A^* + B_i)$, were W_i and B_i are learnable parameter matrices (vectors) and σ is a nonlinearity. The GCNN extends a feed forward neural network by adding an adjacency matrix which premultiplies the node embedding characteristic matrix. This allows the characteristics to effect nearby nodes characteristics more strongly than far away nodes. The inductive bias of GCNNs due to the use of the adjacency matrix allows the model to efficiently handle data defined on networks. For example, a randomly initialized graph neural network can nearly learn how to properly cluster network data. The below graph shows the karate club data classified by color using a modularity-based algorithm (Brandes et al., 2007):



Karate club graph, colors denote communities obtained via modularity-based clustering (Brandes et al., 2008).

Below shows the same cluster colors produced by a randomly initialized graph neural network:

As you can see, the graph convolutional neural network, even before being trained, is already providing a numerical embedding that sorts different clusters into different areas in Euclidian space. With training, one can learn even more effective embeddings that can take a graph and produce sufficient statistics for the simulation-based inference approach. Given the normalizing flow trained to estimate $P(\theta|x)$ and a graph G, the GCNN takes G and returns an numerical embedding x that



GCN embedding (with random weights) for nodes in the karate club network.

can be fed into the normalizing flow to properly condition on the graph.

II.C. FEATHER Graph Embedding

An alternative to using a graph neural network to learn a good embedding is to use an out of the box graph embedding technique and without learning the optimal parameters. The FEATHER algorithm (Rozemberczki and Sarkar, 2020) is the graph embedding algorithm used in this paper. It learns node embeddings from the characteristic function of the pdf implied by a random walk from a node. Then to learn graph embeddings it averages the node embeddings.

For each node, the FEATHER algorithm attempts to learn the characteristic function for the probability distribution of a random walk starting from the source node. Given a graph G with vertex and edges V, E, the FEATHER algorithm learns the function:

$$E[e^{i\theta x}|G, u] = \sum_{w \in V} P(w|u)(\cos(\theta x_w) + i \sin(\theta x_w))$$

Here $P(w|u)$ is the probability that node w is reached from node u after r random walk steps. x_w are the node level features. As my graphs have no node features, the features are the default features of eccentricity, transitivity and degree. One can evaluate the random walk on a graph for r iterations from 1 to n steps from the origional node. Then one can also evaluate the characteristic function at designated points of θ. In order to get a graph level embedding from these node embeddings, the algorithm averages across node embeddings.

One is not limited to FEATHER. Ther are many other approaches used. Other graph embed-

dings approaches include graph2vec (Narayanan et al., 2017), wavelets-based embeddings (Wang et al., 2021), and many others from, for example, the Karate Club library (Rozemberczki, Kiss and Sarkar, 2020). The reason I will use a fixed embedding is that unless one is dealing with sparse graphs the amount of memory required to store a graph is quadratic in the number of nodes. Since SNPE often requires one hundred thousand or more graphs for estimation, one quickly runs out of memory. However with a embedding method without any modifiable weights, after the embedding has been derived, one can throw away the graph and just save the embedding for future iterations. Since the embedding may be 16, 32, or 64 floating number dimensional, this allows one to work with much larger graphs without running into memory constraints.

II.D. Limitations of Graph based Embedding Methods

This section discusses the main weakness of GCNNs, related techniques like graph transformers, and graph embedding methods. One main result discussed in Fen (2022) concern universal approximator properties of flows. The graph SNPE algorithm does not have this quality. Likewise large social networks are also beyond the computational capicity of this GCNNs, although with fixed embeddings work on them is likely possible.

The results in Fen (2022) and borrowed from Huang et al. (2018) demonstrate that the normalizing flow in the SNPE algorithm is a universal approximator of probability distributions and that a large enough normalizing flow can model any continuous probability distribution. However, this result no longer holds with graph data. The reason is because GCNNs, their cousins graph attention transformers (Veličković et al., 2017), and related embedding methods, can not distinguish between certain graphs that are non-isomorphic (Xu et al., 2018). Thus they are not a universal approximator.

Although the result of universal approximation no longer holds, graph convolutional neural networks are a powerful model that have useful inductive biases for embedding graphs. While the results in Papamakarios and Murray (2016) suggest that SNPE converges to the Bayesian posterior, the assumption that the conditioning variable captures all variability in the data is not met when using most graph based embedding models–be they GCNNs or embedding algorithms. Nevertheless,

these approaches are specifically tailored towards graphs and give much better performance on graphs that a feed forward network which have universality guarantees but are not structured for graph based analysis.

Additionally, since the approach relies upon simulating many copies of the graph in question given different structural parameter values, networks that are too large will quickly lead to out of memory errors. In practice this approach still works with models with 9000+ nodes and dense edge connections on a single computer, but requires a relatively small batch size when training the flow.

III. Results

This section produces results demonstrating that the algorithm can recover parameters of a structural model given the graph generated by the parameters in question. First I discuss the Newman-Watts-Strogatz graph. Then I move to the power law cluster and relaxed caveman graphs. Finally I discuss the extended Barabasi-Albert graph and also discuss issues related to identification and good likelihood estimation procedure.

In the below experiments, I draw a simulated graph from the structural model given a set of parameters and then attempt to fit the structural model parameters on the given graph. In each of the charts below, the blue line corresponds to the posterior of the estimation routine, while the red line is the true parameter value. The off diagonal charts show a heat map of the two way distribution implied by the parameter that defines the row and the column that intersects on a given chart. In the case of integer parameters, I still sampled from a continuous uniform distribution, but just applied a floor function to generate an integer. Since the number of nodes is easily established in both the data and models, the model always generates a graph with the same number of nodes as the dataset in question.

III.A. Newman-Watts-Strogatz Small World Graph

I will discuss the results for the small world graph (Newman and Watts, 1999) first. This model has two parameters, k and p. First the number of nodes are arranged in a ring and each node is

connected to the k nearest neighbors in the ring. Then for each edge there is a probability p that the terminating node u of that edge u,v will form a new edge with a random node w. A graph convolutional neural network is used as an graph embedding algorithm. For the generated graph, the parameter k is set to 12 and p is set to .33. As you can see the algorithm recovers a posterior whose mode is quite close to 12 and .33.

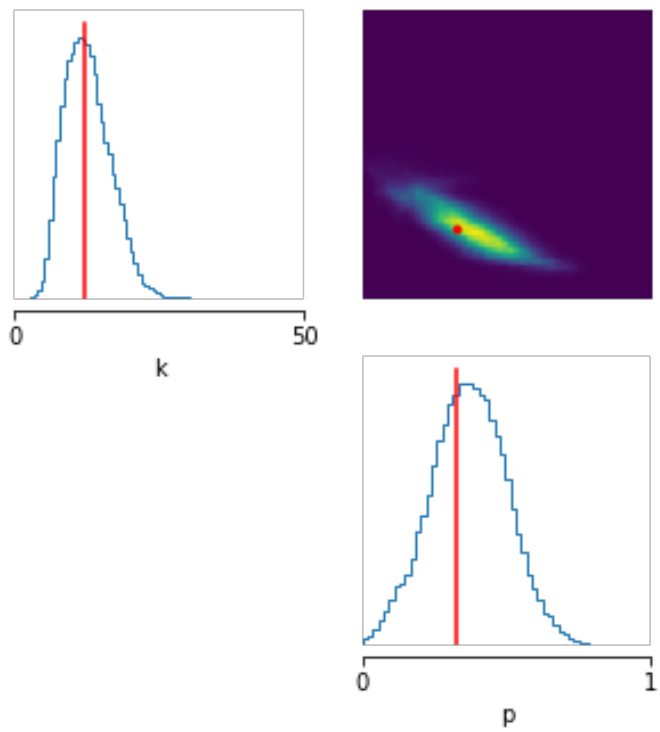


Figure I: Newman-Watts-Strogatz Small World Graph

III.B. Power Law Cluster Graph

Next I will discuss estimating the power law cluster graph (Holme and Kim, 2002) on simulated data. The model has two parameters: the number of random edges to add to each node and the probability of closing a triangle after an edge is added. The random edge parameter has to be an integer, so uniform random variables are generated and then rounded to the nearest integer. The random edge parameter is set to 3, and the triangle probability is set to .35. This results in fewer

triangles being created as the parameters are on the low side. There is good identification of the edge parameter, but the triangle parameter is less well identified in line with the the Exponential Random Graph Model (ERGM) literature, where it is widely known that triangles are difficult to identify (Handcock et al., 2020). A graph convolutional neural network is used as the embedding in this case.

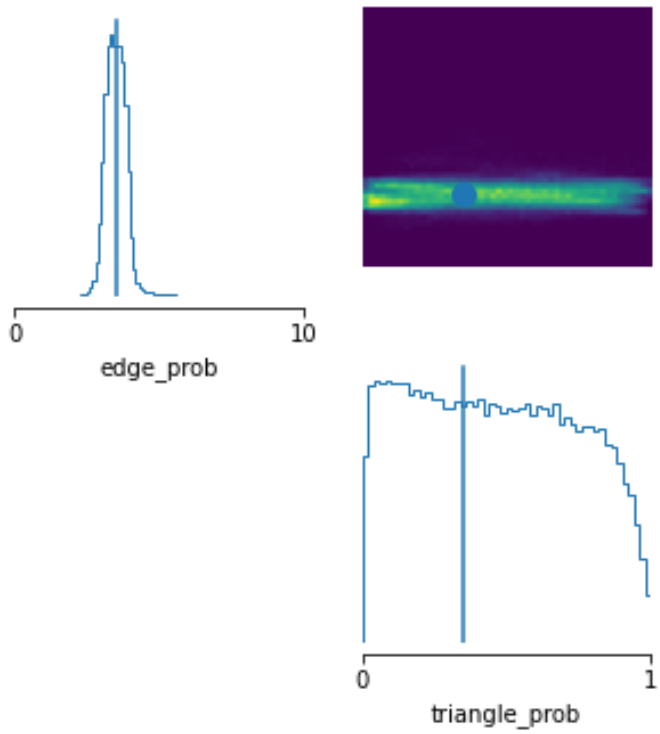


Figure II: Power Law Cluster Graph

This chart shows the model accurately recovering the random edge probability of 3, However there is difficulty recovering the triangle probability as mentioned above.

III.C. Relaxed Caveman Graph

Next I discuss the relaxed caveman graph, formed by randomly rewiring an edge formed by a clique in a caveman graph with probability p (Watts, 2004). The caveman graph is formed by

generating n cliques of size m and then deleting one edge in each clique to form another edge which forms a cycle connecting all the cliques. I use 10 cliques of size 10 with a rewiring probability of 60 percent. The large rewiring probability is used to make the graph look less distinctive and resemble graphs seen in nature more often. I also use the FEATHER graph embedding Rozemberczki and Sarkar (2020) instead of a graph convolutional neural network.

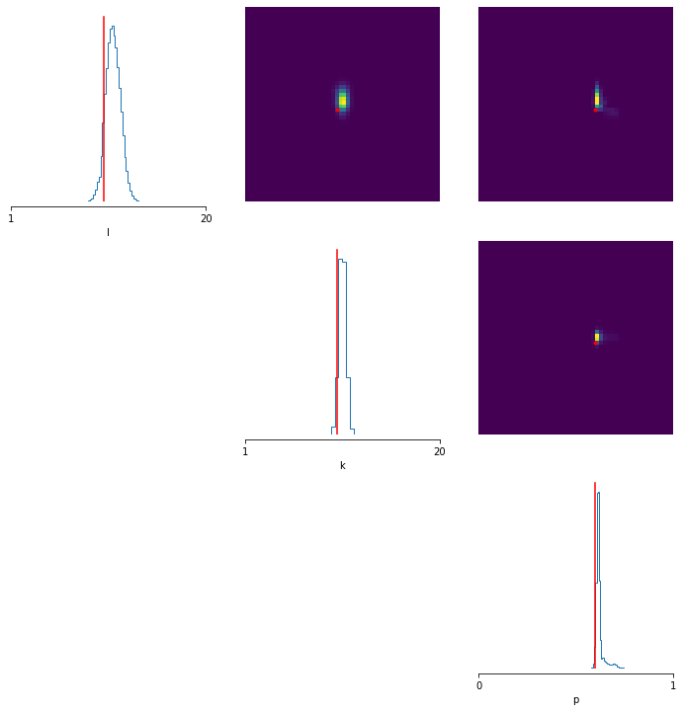


Figure III: Relaxed Caveman Graph

With the relaxed caveman graph, you can see the the true parameters are very close to the mode of the posterior distribution, suggesting good convergence of the algorithm. However it does seem like the modes are slightly shifted from the true value which is a mistake that can often be made using SNPE. Whereas Metropolis Hasting MCMC for Bayesian posterior estimation often has poor mixing problems, with SNPE, because the algorithm relies on sampling and density estimation, often makes mistake like the one above where the entire distribution is shifted slightly from the correct location.

III.D. Extended Barabasi-Albert Graph

Finally I discuss the extended Barabasi-Albert graph (Albert and Barabási, 2000) which is more of a demonstration of the failure case of this algorithm as this approach does sometimes fail to provide good Bayesian estimates. The extended Barabasi-Albert has three parameters, m, p, q. Nodes are added sequentially. m is the number of edges a new edge forms to existing nodes. There are three options, either no edges are formed with probability $1 - p - q$. With probability p, m edges are formed connecting the new node to existing nodes. With probability q, m edges are rewired to terminate at the new node. m is set to 3, p is set to .1 and q is set to .2. For graphical simplicity, p and q range from 0 to .5 for display. This result also uses the FEATHER embedding instead of a graph neural network to generate sufficient statistics to condition the normalizing flow.

In the space of graphs, in order for the likelihood estimation procedure to work, there must be a probability that any graph can be generated from any parameter setting (call this reach-ability), but like the normal distribution, structural parameters that are further away from the data will generate the data less often (call this a ranking). Adding a noise like term like a rewiring probability or a perturbation of the embedding values is not a perfect technique, but the Barabasi-Albert graph shows the issue that arises without a reach-ability parameter. If the number of edges in the data are in the support of the structural model, going through each edge and randomly rewiring the edges allows for the two properties reach-ability and a ranking are fulfilled. The reader should notice, all the previous graphs also have a rewiring probability inherent in the model and not added in. The next chart will show what happens when the rewiring which is not inherent in a extended Barabasi-Albert graph is not added:

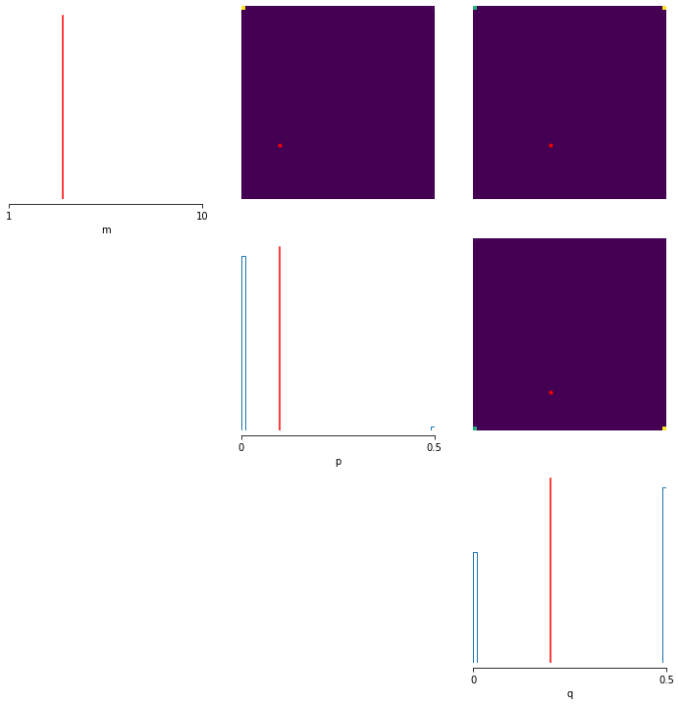


Figure IV: Extended Barabasi-Albert Graph without Perturbations

Next follows a description with intuition provided of cases where estimation fails and why. Since the model output is almost entirely non-overlapping with the generated data other than the knifes edge case where the parameters of the model are the exactly same as the data generating process, the model doesn't learn a posterior as all samples exhibit no likelihood assigned to the data. The analogue to this graph is the example with numerical data where the output has no shocks. The true data has to exactly lie on the the lower dimensional manifold of the model. Since this has a measure zero likelihood of happening, the algorithm cannot rank the parameters likelihood as no parameterization is ever able to generate the data. This chart illustrates the important requirement that an algorithm that calculates the likelihood of the model, even implicitly, has to have a innovation like term that allows for all models to be ranked and reachable by the simulated model.

A second issue with a structural model that prevents identification is building a structural model

that attempts to reproduce global network properties. I contrast this with local network properties like degree number that differ across each node or edge. For example the number of clusters in a graph is a global property. While in some cases estimating these models work–for example the Relaxed Caveman graph has cliques and stochastic block models (Holland, Laskey and Leinhardt, 1983) estimate well–I have found that models estimating these global properties on a single graph often fail.

IV. Empirical Application: Citation Graphs

Next I show an empirical section applying this algorithm on real data. The empirical application is based on Bramoullé et al. (2012) citation graph homophily paper. I will use my algorithm to extend the results of the Bramoullé et al. (2012) paper, which does not structurally estimate their model.

Next I will discuss the model used in the paper. They do not estimate their model on the real data, instead they use cites and cites of cites numbers to come up with statistics that approximate what the true underlying parameter value is. The paper assumes that n nodes are born one after another. Each node that is born sends $m > 1$ connections to previously born nodes. A fraction of these links m_r are connections at random, a fraction of these links $m_s = m - m_r$ are search links–connections formed by citing a paper cited by a paper you cited.

While this approach preserves the temporal nature of citations, the hierarchical nature of this model means that almost for sure the first node born will have the most cites, the second node will have second most, up until 20 or 30 nodes where the randomness effect can take over. When using a likelihood approach, it is essential that the simulated model can generate the real data. Because this hierarchical structure almost always has a particular structure for the papers with the most cites, the model will put zero likelihood on the real world data being generated from this model. This causes the parameters to be unidentified.

Since they don't provide estimation procedure for their approach, they use statistics from the graph to approximate the random connection and search connection rate. They estimate search

homophily probability as any citation that is cited but also has an intermediary that is also cited by and cites the search citation. The remaining cites are classified as random cites. If the model is true this would over count search citations, as there could be some citations that are randomly created even though there is an intermediary that is cited by the citing paper and cites the cited paper. Likewise in the real world this could also under count because someone could cite the paper knowing an intermediary paper and not cite the intermediary paper. This can be approximated by the model if the estimated model determines that it is more accurate to model a cite of a cite even if the original intermediary cite is missing. Estimating a homophily model and perhaps estimating a more complex homophily model that has missing intermediate cites, can take into account these deficiencies.

I modify the Bramoullé et al. (2012) model in the following ways. All nodes are initialized at the beginning. At the beginning of each round, there is a probability that each node has a direct connection with another node. Additionally, there is a separate probability that a node is connected to the connection of a connection. Since temporal relationships have been destroyed, this is an undirected network with the node representing a citation from the earlier paper to the later paper. There are 15 rounds in total.

There is a slight issue with timing of citations using an undirected graph. Random cites have little issue. The direction of each undirected cite flows from the later paper to the earlier. However it may not be clear that this convenience will work for a search citation. For example two people can cite the same paper that happens later than both of them. But one of the paper also has a search citation and cites the third paper that may have been published earlier than the intermediary. This can be resolved geometrically by choosing the correct citation as the search citation. For example, a search citation along with the direct citation will form a triangle. In this triangle, there is always a direct path going from youngest node to oldest node and an indirect path going from youngest to oldest through the intermediate. It may not be the actual search link, but the number of direct citations and the number of search citations will still be the same. That being said, in the introductory example of friendships the choice of using undirected links makes more direct sense and one doesn't have to do mathematical cartwheels to justify the links.

IV.A. Karate Club Friendships Graph

First, I estimate the model on the karate club dataset. This modification allows for identification as the empirical data can be generated by this structural model. This is a dataset of friendship connections between karate club participants. The model hypothesizes that one makes direct friends and then one makes friends from friends of ones friends. First I get the following posterior:

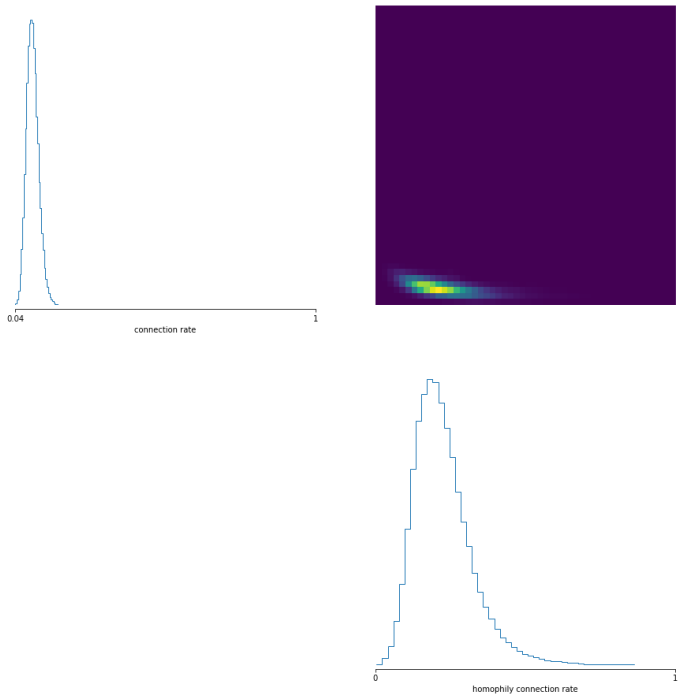


Figure V: Homophily Model estimated on Karate Club data

Next to verify that the model is converging to the true solution, I estimate the model on data generated by the model where the random connection rate is set to .08, and the homophily search connection rate is set to .23. These were the approximate means of the empirical posterior. Here is the estimation procedure recovering the true parameters on the simulated data:

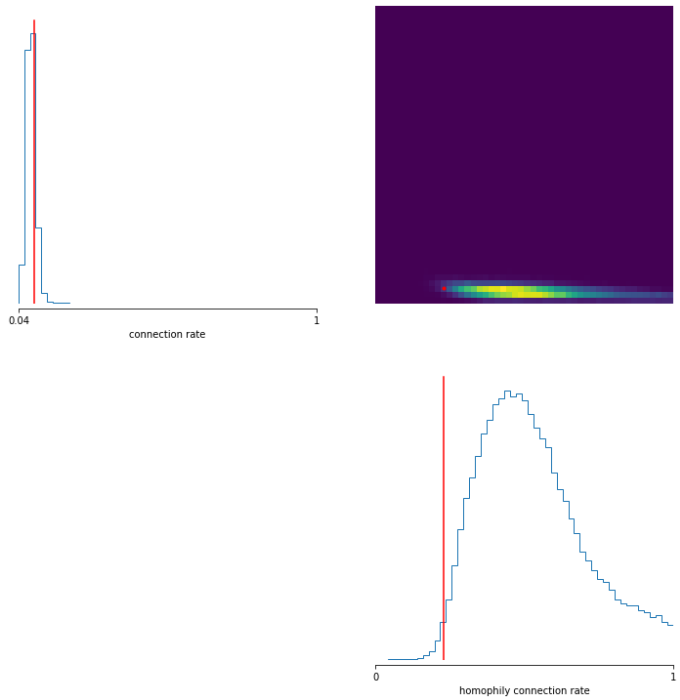


Figure VI: Homophily Model estimated on Homophily simulated data

As you can see from the pictures, the model does a good job at identifying the random connection parameters. The model does a poorer job of identifying the homophily search paramater. I attribute this to the fact that the simulated data–in line with the karate club data–has only 34 nodes and so some there will be some variance in the simulated data. Nevertheless, the true parameter is still within the support of the posterior for the homophily search parameter.

IV.B. American Physical Society Citations

Next I will estimate my model on the American Physical Society (APS) citation data used in Bramoullé et al. (2012). Using the method of counting cites and cites of cites, Bramoullé et al. (2012) relay that 56.34 percent of cites are search homophily cites and 43.66 percent of cites are random cites, the astrophysics sector which I used to estimate my structural model has x search cites and y random cites. The posterior of the cites model estimated on the citation network has

this posterior:

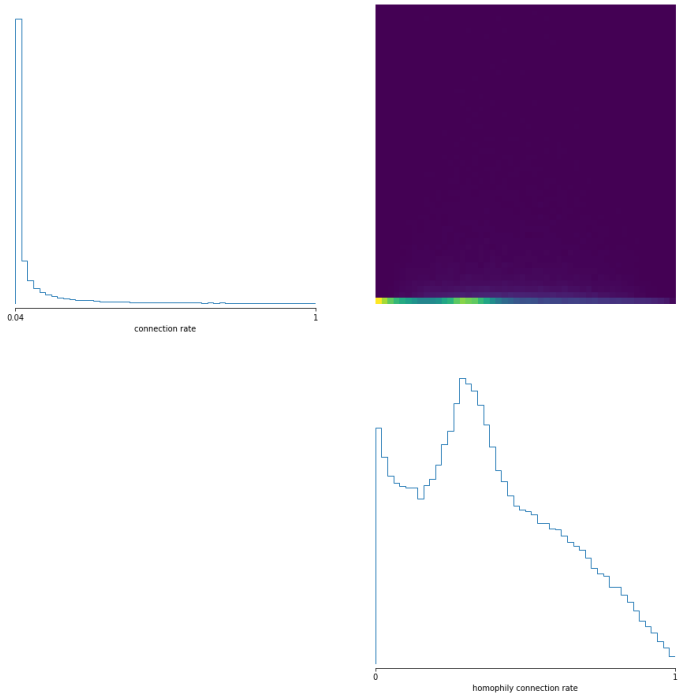


Figure VII: Homophily Model estimated on APS data

At first glance it looks like there were a lot more search cites than homophily cites. However, this is partially an artifact of the structural model. Since the random and search cites are performed in an alternating manner, if there are no random cites that a network has (and the number of random cites forming ranges between a 4 percent chance to an 8 percent chance each turn), then regardless of the probability of the search cites, no search cite will occur. Incidentally this may contribute to the less than stellar identification issues with the search homophily connection rate. To truly estimate the ratio of random to search cites, estimated a model of each posterior parameter value and constructed a simulated citation network. While constructing the network I recorded the ratio of random cites to search cites:

Figure VIII: Graph that's not done yet

The mode of this graph shows .04 random cites and .18 search cites. Inputting the modal numbers into the structural model and counting the number of each type of citation yields 45.71 percent random cites and 54.29 percent search cites. The fact that these maximum likelihood values adhere closely to the counting method is comforting as the most likely values should be approximately the values of counted search and random connections. However, using the mean of the random and search cites suggest a different story. Here the mean number of random cites implies that 29.41 percent of links are random and 70.59 percent are search. This suggests there were a lot more search cites that missed citing the intermediary paper. This is something that the Bramoullé et al. (2012) technique could not identify by just counting the links. This is perhaps not surprising as most researchers know about the seminal papers in their field and may not bother or may not be able to cite the resource that first pointed out that paper. The empirical section of this paper advances this literature by providing this additional modeling and structural identification of parameters behind the process of citations.

References

Agarap, Abien Fred. 2018. "Deep learning using rectified linear units (relu)." *arXiv preprint arXiv:1803.08375*.

Albert, Réka, and Albert-László Barabási. 2000. "Topology of evolving networks: local events and universality." *Physical review letters*, 85(24): 5234.

Banerjee, Abhijit, Arun G Chandrasekhar, Esther Duflo, and Matthew O Jackson. 2013. "The diffusion of microfinance." *Science*, 341(6144): 1236498.

Bramoullé, Yann, Sergio Currarini, Matthew O Jackson, Paolo Pin, and Brian W Rogers. 2012. "Homophily and long-run integration in social networks." *Journal of Economic Theory*, 147(5): 1754–1786.

Brandes, Ulrik, Daniel Delling, Marco Gaertler, Robert Gorke, Martin Hoefer, Zoran Nikoloski, and Dorothea Wagner. 2007. "On modularity clustering." *IEEE transactions on knowledge and data engineering*, 20(2): 172–188.

Calvo-Armengol, Antoni, and Matthew O Jackson. 2004. "The effects of social networks on employment and inequality." *American economic review*, 94(3): 426–454.

Calvó-Armengol, Antoni, and Matthew O Jackson. 2007. "Networks in labor markets: Wage and employment dynamics and inequality." *Journal of economic theory*, 132(1): 27–46.

Carvalho, Vasco M, and Nico Voigtländer. 2014. "Input diffusion and the evolution of production networks." National Bureau of Economic Research.

Fen, Cameron. 2022. "Fast Simulation-Based Bayesian Estimation of Dynamic Models using Normalizing Flow Neural Networks."

Furusawa, Taiji, and Hideo Konishi. 2007. "Free trade networks." *Journal of International Economics*, 72(2): 310–335.

Gilles, Robert P, Cathleen Johnson, et al. 2000. "original papers: Spatial social networks." *Review of Economic Design*, 5(3): 273–299.

Handcock, Mark S, David R Hunter, Carter T Butts, Steven M Goodreau, Pavel N Krivitsky, and Martina Morris. 2020. "ergm: Fit, Simulate and Diagnose Exponential-Family Models for Networks."

Holland, Paul W, Kathryn Blackmond Laskey, and Samuel Leinhardt. 1983. "Stochastic blockmodels: First steps." *Social networks*, 5(2): 109–137.

Holme, Petter, and Beom Jun Kim. 2002. "Growing scale-free networks with tunable clustering." *Physical review E*, 65(2): 026107.

Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. 1989. "Multilayer feedforward networks are universal approximators." *Neural networks*, 2(5): 359–366.

Huang, Chin-Wei, David Krueger, Alexandre Lacoste, and Aaron Courville. 2018. "Neural autoregressive flows." 2078–2087, PMLR.

Jackson, Matthew O, and Asher Wolinsky. 1996. "A strategic model of social and economic networks." *Journal of economic theory*, 71(1): 44–74.

Kipf, Thomas N, and Max Welling. 2016. "Semi-supervised classification with graph convolutional networks." *arXiv preprint arXiv:1609.02907*.

Kristensen, Dennis, and Yongseok Shin. 2012. "Estimation of dynamic models with nonparametric simulated maximum likelihood." *Journal of Econometrics*, 167(1): 76–94.

McFadden, Daniel. 1989. "A method of simulated moments for estimation of discrete response models without numerical integration." *Econometrica: Journal of the Econometric Society*, 995–1026.

Narayanan, Annamalai, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. 2017. "graph2vec: Learning distributed representations of graphs." *arXiv preprint arXiv:1707.05005*.

Newman, Mark EJ, and Duncan J Watts. 1999. "Renormalization group analysis of the small-world network model." *Physics Letters A*, 263(4-6): 341–346.

Papamakarios, George, and Iain Murray. 2016. "Fast ε-free inference of simulation models with bayesian conditional density estimation." *Advances in neural information processing systems*, 29.

Rozemberczki, Benedek, and Rik Sarkar. 2020. "Characteristic functions on graphs: Birds of a feather, from statistical descriptors to parametric models." 1325–1334.

Rozemberczki, Benedek, Oliver Kiss, and Rik Sarkar. 2020. "Karate Club: An API Oriented Open-source Python Framework for Unsupervised Learning on Graphs." 3125–3132, ACM.

Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. "Graph attention networks." *arXiv preprint arXiv:1710.10903*.

Wang, Lili, Chenghan Huang, Weicheng Ma, Xinyuan Cao, and Soroush Vosoughi. 2021. "Graph Embedding via Diffusion-Wavelets-Based Node Feature Distribution Characterization." 3478–3482.

Watts, Duncan J. 2004. *Small worlds: the dynamics of networks between order and randomness.* Vol. 36, Princeton university press.

Xu, Keyulu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. "How powerful are graph neural networks?" *arXiv preprint arXiv:1810.00826*.

Department of Economics, University of Michigan, Ann Arbor